

# SYSTÈMES RAG ET BASES DE DONNÉES — retrieval, embeddings, vector stores, RAG pour vos docs

---

> Promesse mesurable : à la fin, vous avez construit un système RAG fonctionnel qui répond avec VOS documents (PDF, notes, emails, wiki) — avec une base vectorielle déployée, un pipeline d'indexation et de recherche, et une évaluation chiffrée de sa qualité. Fini les hallucinations sur vos données : chaque réponse est sourcée.

>

> Format : pack PDF, 7 modules, ~6 h de lecture + travaux pratiques.

> Cible : indépendants, techniciens, chefs de projet et créateurs qui veulent faire parler leurs documents à un LLM (GPT, Claude, Llama) — sans dépendre d'un service opaque.

> Prérequis : savoir exécuter un script Python et lire un terminal. La formation 1 (LLM) est recommandée ; les formations 2 et 3 (agents, prompt) en font un complément naturel.

> Durée : 6 h sur 2 semaines, dont 3 h de pratique (installation, code, tests).

## Module 1 — Pourquoi votre LLM ne connaît pas vos documents (et ce que le RAG change)

### Leçon

Un LLM est entraîné sur un corpus public coupé à une date donnée. Il ne connaît ni vos contrats, ni votre wiki interne, ni le dernier PDF de votre secteur. Et quand il répond quand même, il hallucine : il invente une réponse plausible — c'est sa spécialité.

Il y a 4 façons de donner des connaissances à un LLM :

| Méthode | Principe | Quand l'utiliser | Limites |

|---|---|---|---|

| Contexte dans le prompt | Coller les extraits dans la conversation | Petits volumes (< 5 000 mots), une fois | Coût, taille limitée, à refaire à chaque fois |

| Fine-tuning | Ré-entraîner le modèle sur vos données | Styles et formats répétés, pas des faits | Coûteux, se périmé dès que les données changent, surkill pour des faits |

| RAG (Retrieval-Augmented Generation) | Chercher les bons extraits AVANT de répondre, et les donner en contexte | Le standard pour les faits et documents qui changent | Demande une base vectorielle et un pipeline (ce que cette formation construit) |

| Connaissances intégrées à l'outil | Services propriétaires (ex. « uploader ses docs » dans un assistant)

| Démarrage rapide | Données chez un tiers, boîte noire, coûts cachés, impossible à brancher sur vos

process |

Le principe du RAG, en une phrase : au lieu de demander au modèle « réponds », on lui demande « cherche dans mes documents puis réponds avec ce que tu as trouvé ». Le modèle ne répond plus de mémoire — il répond à partir d'extraits qu'on lui fournit, sourcés.

Le pipeline en 4 étapes (à retenir, tout le reste de la formation déplie ces 4 cases) :

|               |                                                                                    |
|---------------|------------------------------------------------------------------------------------|
| 1. INDEXATION | → découper vos documents en morceaux (chunks), les transformer en vecteurs         |
| 2. STOCKAGE   | → ranger les vecteurs dans une base vectorielle                                    |
| 3. RETRIEVAL  | → à la question, transformer la question en vecteur, chercher les morceaux proches |
| 4. GÉNÉRATION | → donner les morceaux trouvés au LLM en contexte, il répond en citant ses sources  |

Pourquoi ça marche : le RAG remplace la mémoire défaillante du modèle par une recherche déterministe dans vos données. Ce que le modèle ne sait pas, on le lui apporte. Résultat : réponses exactes sur vos docs, citations vérifiables, et vos données qui peuvent changer sans retoucher au modèle.

Le piège n°1 : croire que RAG = « uploader un PDF ». Coller un document entier dans le prompt n'est pas du RAG — c'est du contexte, cher et inefficace au-delà de quelques pages. Le RAG est un système de recherche, pas un copier-coller : il doit trouver LE bon paragraphe parmi des milliers, en quelques dizaines de millisecondes.

## Exemple réel

Un cabinet de conseil avec 4 000 pages de rapports de missions. Un collaborateur demande à ChatGPT : « Quelles recommandations avons-nous faites à Dupont Industries ? » → réponse inventée, car les rapports ne sont pas dans le modèle. Avec un RAG : la question est transformée en vecteur, la base retrouve les 3 extraits pertinents des rapports Dupont, le LLM répond avec ces extraits et cite le numéro de page. Vérifiable, exact, à jour.

## Exercice pratique

1. Listez 3 de vos documents que vous aimeriez « faire parler » à un LLM (PDF, notes, exports).
2. Pour chacun, notez : volume approximatif, fréquence de mise à jour, et 5 questions que vous voudriez lui poser.
3. Essayez de poser ces questions à ChatGPT sans vos documents : notez où il hallucine. C'est votre point de départ (et votre futur démo).

# Module 2 — Les embeddings : transformer des textes en nombres

## Leçon

Un embedding est une représentation numérique d'un texte : une liste de nombres (par exemple 1 536 pour OpenAI, 1 024 pour Cohere, 384 pour les modèles open source) qui encode le sens, pas les mots. Deux textes de sens proche ont des vecteurs proches ; deux textes sans rapport ont des vecteurs éloignés. C'est la brique fondamentale : sans embeddings, pas de recherche sémantique.

La mesure clé : la similarité cosinus. Elle vaut de -1 (opposés) à 1 (identiques). Pour la recherche : on calcule la similarité entre le vecteur de la question et chaque vecteur stocké, on garde les top-k (souvent 3 à 10).

Le choix du modèle d'embedding (décision structurante) :

| Modèle | Taille de vecteur | Coût/1M tokens (API) | Usage conseillé |

|---|---|---|---|

| text-embedding-3-small (OpenAI) | 1 536 | ~0,02 \$ | Standard, très bon rapport qualité/prix |

| text-embedding-3-large (OpenAI) | 3 072 | ~0,13 \$ | Qualité maximale, gros volumes |

| embed-multilingual-v3 (Cohere) | 1 024 | ~0,10 \$ | Excellent multilingue, 100+ langues |

| bge-m3 / multilingual-e5 (open source) | 1 024 | 0 (local) | Français OK, zéro coût, données privées |

Règle de bon sens : vos documents sont en français ? Prenez un modèle multilingue ou testez les deux sur VOS données — ne vous fiez pas aux benchmarks.

Le code réel (Python, 10 lignes, API OpenAI — la clé passe par une variable d'environnement, jamais en dur dans le code) :

```
pip install openai
export OPENAI_API_KEY="sk-..." # jamais dans le code, ni dans un repo public
```

```
from openai import OpenAI
client = OpenAI()

def embed(texts: list[str]) -> list[list[float]]:
    r = client.embeddings.create(model="text-embedding-3-small", input=texts)
    return [d.embedding for d in r.data]

# Test : deux phrases proches, une phrase sans rapport
v1 = embed(["Le devis doit partir sous 48 heures"])[0]
v2 = embed(["La proposition commerciale est envoyée rapidement"])[0]
v3 = embed(["Le gâteau est au four depuis dix minutes"])[0]

import math
def cos(a, b):
    return sum(x*y for x, y in zip(a, b)) / (math.sqrt(sum(x*x for x in a)) * math.sqrt(sum(y*y for y in b)))

print(cos(v1, v2)) # ~0,85 : sens proche
print(cos(v1, v3)) # ~0,35 : sans rapport
```

Le piège n°2 : les embeddings ne comprennent pas le contexte long. Un embedding est un résumé statistique du sens global d'un texte. Une question de 500 mots et un paragraphe de 500 mots donnent des vecteurs brouillons. D'où le chunking (module 4) : on n'embed pas un document entier, on embed des morceaux.

## Exemple réel

Une association indexe ses 300 pages de comptes rendus de réunions. Recherche « combien de bénévoles en 2025 ? » → la similarité cosinus remonte les chunks qui parlent de bénévoles et de 2025, même si le mot exact « combien » n'apparaît nulle part. C'est le gain du sémantique sur le mot-clé : une

recherche par mots ne trouve « 2025 » que si la question contient « 2025 ».

## Exercice pratique

1. Installez le SDK et faites tourner le code ci-dessus (3 phrases, 3 vecteurs, 3 similarités).
2. Testez 5 synonymes et périphrases de votre métier : « relance », « suivi client », « rappel de devis »... observez les scores.
3. Notez le coût réel de vos tests : comptez les tokens consommés (le SDK les retourne) et multipliez par le prix. Vous verrez qu'une session de test coûte quelques centimes.

## Module 3 — Les bases vectorielles : ChromaDB, LanceDB, pgvector

### Leçon

Une base vectorielle stocke les vecteurs et sait chercher les plus proches d'un vecteur donné (similarité cosinus, distance euclidienne...). C'est le « classeur » du RAG. Le choix se joue sur 4 critères : simplicité, volume, données existantes, coût.

| Base | Type | Installer | Pour qui |

|---|---|---|---|

| ChromaDB | Base vectorielle dédiée, embarquée | `pip install chromadb` | Le meilleur point de départ : zéro infra, parfaite pour apprendre et pour des volumes < 1 M de chunks |

| LanceDB | Embarquée, orientée fichiers, très rapide | `pip install lancedb` | Données en fichiers Parquet, gros volumes, pas de serveur à gérer |

| pgvector | Extension PostgreSQL | `docker run -d -p 5432:5432 -e POSTGRES_PASSWORD=... pgvector/pgvector:pg17` | Si vous avez déjà PostgreSQL : vos données métier et leurs vecteurs dans la même base, jointures possibles |

| Qdrant / Weaviate / Milvus | Serveurs dédiés | Docker ou cloud | Équipes, multi-projets, échelle ; surkill en solo au début |

La règle d'engagement : commencez par ChromaDB (aucun serveur, un dossier local), et ne migrez vers pgvector que si vous avez déjà PostgreSQL en production. Un RAG qui tourne sur ChromaDB est un vrai RAG — la base n'est pas le produit, le pipeline l'est.

Le code réel avec ChromaDB (indexation + recherche en 15 lignes) :

```
pip install chromadb
```

```
import chromadb
from openai import OpenAI

client = OpenAI()
chroma = chromadb.PersistentClient(path="./ma_base") # tout tient dans un dossier
collection = chroma.get_or_create_collection(
    name="mes_docs",
    metadata={"hnsw:space": "cosine"}, # la métrique de similarité
)
```

```
# Indexer un document découpé en chunks
chunks = ["Le devis part sous 48 h.", "Les relances se font à J+10.", "Le paiement est exigible à réception"]
embeddings = [d.embedding for d in client.embeddings.create(
    model="text-embedding-3-small", input=chunks).data]
collection.add(ids=[f"c{i}" for i in range(len(chunks))],
               documents=chunks, embeddings=embeddings,
               metadatas=[{"source": "process-devis.md"} for _ in chunks])

# Rechercher
q = "Quand dois-je relancer un client ?"
qv = client.embeddings.create(model="text-embedding-3-small", input=[q]).data[0].embedding
hits = collection.query(query_embeddings=qv, n_results=3)
print(hits["documents"])    # les 3 chunks les plus proches
print(hits["distances"])    # leurs scores (plus petit = plus proche)
```

Les métadonnées, votre arme secrète : chaque chunk peut porter des métadonnées (source, date, auteur, client, type). Elles permettent les filtres — « ne cherche que dans les contrats de 2025 » — et l'affichage des sources dans la réponse finale. Un RAG sans métadonnées est un RAG sans sources ni filtres.

Le piège n°3 : comparer les bases sur des benchmarks au lieu de ses données. Le « meilleur » vector store n'existe pas. Testez ChromaDB sur VOS documents, mesurez vos propres scores (module 7), et ne migrez que si vous avez une raison mesurée (volume, jointures SQL, multi-utilisateurs).

## Exemple réel

Un formateur indexe ses supports de cours (50 PDF). ChromaDB dans un dossier ./mabase, un chunk par paragraphe, métadonnées {source: "module-3.pdf", titre: "...". Un stagiaire demande : « c'est quoi un embedding ? » → la recherche remonte les chunks du module 2, le LLM répond en citant « module-2.pdf ». Migration vers pgvector décidée 6 mois plus tard, uniquement parce que le formateur a ajouté un CRM PostgreSQL et veut joindre clients ↔ documents.

## Exercice pratique

1. Installez ChromaDB et lancez le code ci-dessus avec VOS 3 phrases.
2. Ajoutez 10 chunks de l'un de vos vrais documents, avec métadonnées.
3. Faites 3 recherches (une exacte, une en synonymes, une à contresens) et notez les distances : vérifiez que l'ordre est celui que vous attendiez.
4. (Bonus) Si vous avez PostgreSQL : lancez pgvector en Docker et reproduisez le même exemple avec une table chunks(id, content, embedding vector(1536)) et une requête ORDER BY embedding <=> \$1 LIMIT 3.

# Module 4 — Le pipeline RAG complet : indexer, chercher, répondre avec sources

## Leçon

Maintenant on assemble. Le pipeline complet a 3 services : indexeur (une fois par document), chercheur (à chaque question), rédacteur (le LLM avec contexte).

Le chunking — la décision qui fait 80 % de la qualité :

| Stratégie | Comment | Quand |

|---|---|---|

| Fixe par caractères | 500-1 000 caractères, chevauchement 10-20 % | Standard, simple, souvent suffisant |

| Par paragraphes / titres | Découper sur les retours à la ligne, marques markdown, titres | Documents structurés (wiki, rapports, docs) |

| Par sections sémantiques | Découper quand le sens change (changement de sujet détecté) | Gros corpus hétérogènes ; plus complexe |

Règles pratiques : 300-800 caractères par chunk pour du français (plus petit que l'anglais en général), chevauchement de 10-20 % pour ne pas couper une idée en deux, et toujours embarquer le titre de la section dans le chunk (« [Module 2 — Embeddings] Le texte... ») — ça améliore la recherche de 5-10 points, c'est le hack le moins connu et le plus efficace.

Le template de prompt RAG (à copier) :

Tu es un assistant qui répond UNIQUEMENT à partir du contexte fourni.

RÈGLES :

1. Si l'information est dans le contexte, réponds précisément et cite la source entre crochets [Source : X]
2. Si l'information n'est PAS dans le contexte, réponds : « Cette information n'est pas dans les documents »
3. Ne mélange pas le contexte avec tes connaissances générales.
4. Réponds en français, directement, sans introduction.

CONTEXTE :

{extraits\_retrouvés}

QUESTION : {question}

Le point 2 est le garde-fou anti-hallucination : le modèle a le droit de dire « je ne sais pas » — c'est ce qui rend un RAG fiable.

Le code réel — le pipeline complet en 40 lignes (Python, fichier rag.py) :

```
import os, sys
import chromadb
from openai import OpenAI

client = OpenAI()
chroma = chromadb.PersistentClient(path="./ma_base")
collection = chroma.get_or_create_collection("mes_docs", metadata={"hnsw:space": "cosine"})

def decouper(texte: str, taille=600, chevauchement=100):
    chunks, i = [], 0
    while i < len(texte):
        chunks.append(texte[i:i+taille])
        i += taille - chevauchement
    return chunks
```

```
def indexer(chemin: str):
    texte = open(chemin, encoding="utf-8").read()
    chunks = decouper(texte)
    emb = [d.embedding for d in client.embeddings.create(
        model="text-embedding-3-small", input=chunks).data]
    collection.add(ids=[f"{chemin}#{i}" for i in range(len(chunks))],
        documents=chunks, embeddings=emb,
        metadatas=[{"source": os.path.basename(chemin)} for _ in chunks])
    print(f"Indexé : {chemin} → {len(chunks)} chunks")

def repondre(question: str, k=5):
    qv = client.embeddings.create(model="text-embedding-3-small", input=[question]).data[0].embedding
    hits = collection.query(query_embeddings=[qv], n_results=k)
    extraits = "\n\n".join(hits["documents"][0])
    sources = sorted({m["source"] for m in hits["metadatas"][0]})
    reponse = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "system",
            "content": "Tu réponds UNIQUEMENT à partir du contexte. Sinon, dis que l'info n'est pas"},
            {"role": "user", "content": f"CONTEXTE : \n{extraits}\n\nQUESTION : {question}"}])
    return reponse.choices[0].message.content, sources

if __name__ == "__main__":
    if sys.argv[1] == "index":
        indexer(sys.argv[2])
    else:
        rep, sources = repondre(sys.argv[2])
        print(rep)
        print("\nSources :", ", ".join(sources))
```

```
python rag.py index contrat.pdf # indexe
python rag.py "Quelle est la durée du préavis ?" # répond avec sources
```

Le coût réel d'une question : ~0,001 € (embedding question + ~2 000 tokens de contexte) avec gpt-4o-mini. C'est le modèle économique qui rend le RAG viable en solo.

Le piège n°4 : livrer le RAG sans le prompt « je ne sais pas ». Sans la règle 2, le modèle compense les trous du contexte avec ses connaissances — et vous croyez avoir un système fiable alors que vous avez un LLM déguisé. La fiabilité d'un RAG se joue dans le prompt, pas dans la base.

## Exemple réel

Un consultant indexe le guide des procédures de son client (120 pages, PDF). Question réelle posée ensuite : « Que faire si un salarié part en arrêt maladie le premier jour de sa période d'essai ? » → le RAG remonte les 2 sections concernées (période d'essai + arrêt maladie), le LLM synthétise en citant les pages. Question piège : « Quelle est la politique télétravail ? » (absente du guide) → « Cette information n'est pas dans les documents fournis. » C'est exactement la bonne réponse.

## Exercice pratique

1. Reprenez votre fichier rag.py et indexez 3 vrais documents.
2. Posez 10 vraies questions : 5 dont vous connaissez la réponse (vérifiez l'exactitude), 3 dont vous savez qu'elles sont hors documents (vérifiez le « je ne sais pas »), 2 transverses (multi-sources).

3. Vérifiez que chaque réponse cite la bonne source. Corrigez le chunking si un chunk est tronqué en plein milieu d'une idée.

## Module 5 — RAG avancé : recherche hybride, reranking, filtres — la qualité pro

### Leçon

Le RAG de base fonctionne. Le RAG pro se distingue sur 4 leviers, dans l'ordre d'impact :

1. La recherche hybride (le plus gros gain). Les vecteurs sont excellents pour le sens, mauvais pour les termes précis (« référence contractuelle AX-2024-17 », un numéro de pièce, un nom propre rare). La recherche BM25 (classique, par mots) est l'inverse. L'hybride = faire les deux et fusionner les résultats (méthode RRF : on classe par rang, pas par score). Un numéro de pièce est retrouvé par BM25 ; une question sémantique par les vecteurs ; ensemble, ils se couvrent.
2. Le reranking. Après la première recherche (50-100 candidats), un modèle de reranking (ex. cohere-rerank, ou open source bge-reranker) re-score les candidats en comparant la question et chaque chunk — plus précis que la similarité brute. On ne garde que les top-5. Gain typique : +5-10 points de qualité. Coût : un appel API par question, quelques dixièmes de centime — c'est le meilleur investissement qualité/prix du RAG.
3. Les filtres par métadonnées. Restreindre la recherche : {"client": "Dupont"}, {"date": {"\$gte": "2025-01-01"}}. Indispensable dès que le corpus dépasse quelques milliers de chunks : on cherche dans le bon tiroir avant de chercher.
4. Les questions de suivi et la mémoire. « Et pour les congés ? » sans contexte n'a pas de sens. On reformule la question avec l'historique avant la recherche (une étape LLM) — ou on indexe le résumé de la conversation.

Le code réel — hybride + rerank avec ChromaDB et Cohere :

```
pip install chromadb cohere
export CO_API_KEY="..." # clé Cohere pour le reranking

import chromadb, cohere
from openai import OpenAI

co = cohere.Client()
oa = OpenAI()
chroma = chromadb.PersistentClient(path="./ma_base")
collection = chroma.get_or_create_collection("mes_docs", metadata={"hnsw:space": "cosine"})

# 1. Recherche vectorielle élargie (top 50, pas top 5)
q = "Quelles sont les étapes de facturation ?"
qv = oa.embeddings.create(model="text-embedding-3-small", input=[q]).data[0].embedding
candidats = collection.query(query_embeddings=qv, n_results=50,
                             where={"date": {"$gte": "2025-01-01"}}) # filtre métadonnées
docs = candidats["documents"][0]
```



```
# 2. Reranking : la question re-score les 50 candidats, on garde les 5 meilleurs
r = co.rerank(query=q, documents=docs, top_n=5, model="rerank-multilingual-v3.0")
meilleurs = [c.document["text"] for c in r.results] # triés par pertinence

# 3. Génération avec les 5 meilleurs (même prompt que le module 4)
```

L'ordre d'implémentation conseillé : 1) filtres métadonnées (gratuit, immédiat), 2) hybride (quand les termes précis manquent), 3) reranking (quand on veut le dernier point de qualité), 4) mémoire (quand les utilisateurs enchaînent les questions).

Le piège n°5 : empiler les techniques sans mesure. Hybride + rerank + mémoire = complexité. Si vos erreurs viennent des chunks mal découpés, aucun reranker n'y remédiera. Mesurez d'abord (module 7), améliorez ensuite — chaque ajout doit se justifier par un score qui bouge.

## Exemple réel

Un expert-comptable indexe 2 ans d'échanges (emails, notes, décisions). Question : « Le client Martin a-t-il validé le devis AX-2024-17 ? » → la recherche vectorielle seule rate la référence exacte (le sens du devis n'est pas « devis ») ; BM25 la retrouve par le numéro ; le reranker place l'email de validation en tête ; la réponse cite l'email, la date, l'auteur. Le tout pour ~0,002 € par question.

## Exercice pratique

1. Ajoutez des métadonnées date à vos chunks et refaites une recherche avec filtre.
2. Intégrez le reranking Cohere (le code ci-dessus) sur vos 10 questions du module 4 : comparez les réponses avant/après.
3. Identifiez 3 questions « à terme précis » de votre domaine (numéros, références, noms propres) et vérifiez que l'hybride les retrouve.

# Module 6 — RAG en production : n8n, coûts, mise à jour, sécurité

## Leçon

Un RAG qui tourne une fois est un exercice. Un RAG qui tourne en continu est un service. Trois chantiers : brancher le monde réel (n8n), maîtriser les coûts, sécuriser.

1. Brancher le RAG sur vos outils avec n8n (la stack de la formation 2). Trois workflows qui couvrent 90 % des besoins :
  - Indexation automatique : déclencheur « Nouveau fichier » (Google Drive, dossier) ou « Nouvel email » → n8n lit le document, le découpe, calcule les embeddings (nœud OpenAI Embeddings) → écrit dans ChromaDB (nœud HTTP vers l'API locale) ou pgvector.
  - Chat avec sources : webhook Telegram/Slack → n8n reçoit la question → recherche les chunks (HTTP vers votre API) → envoie le contexte au LLM → répond avec les sources.
  - Rafraîchissement périodique : cron (ex. chaque lundi 6 h) qui re-indexe les documents modifiés depuis la dernière passe.

2. La mise à jour des données — la question qui tue. Une base vectorielle ne « sait » pas qu'un document a changé. Deux stratégies simples : ré-indexer par lot (supprimer les chunks du document puis ré-indexer — le plus fiable), ou versionner par date (chunks avec métadonnée date, filtre date >= ...). Un RAG avec des données périmées est pire qu'un RAG absent : il répond avec assurance sur des informations fausses.

3. Les coûts réels (à chiffrer avant de déployer) :

| Poste | Ordre de grandeur |

|---|---|

| Indexation | 1-3 € par million de tokens de documents (embedding small) |

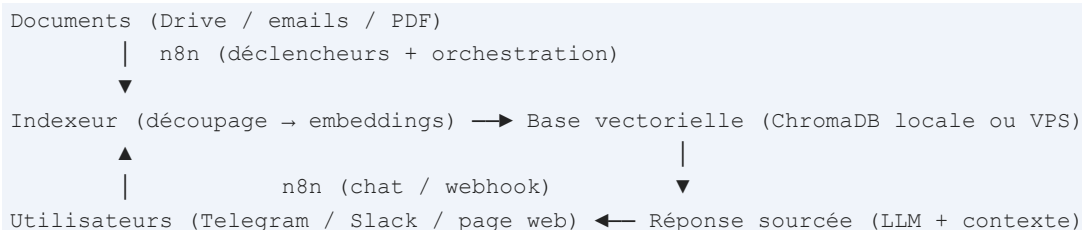
| Question type | 0,001-0,003 € (embedding + 2-5 k tokens de contexte + réponse) |

| Reranking | +0,0002-0,001 € par question |

| Hébergement (ChromaDB local) | 0 € (votre machine) ou ~5-10 €/mois (petit VPS) |

4. La sécurité — non négociable : les clés API vivent dans des variables d'environnement ou un coffre (Keychain, Docker secrets), jamais dans du code versionné ni dans une page web. Si le RAG est exposé sur le web, passez par une passerelle (n8n ou Cloudflare Workers) : l'utilisateur parle à la passerelle, la clé ne quitte jamais votre serveur. Et posez-vous la question des données : vos documents contiennent-ils des données personnelles ? Si oui, chiffrement au repos, accès restreint, et mention dans votre registre RGPD (un RAG sur des données clients = un traitement de données).

5. L'architecture cible en solo :



## Exemple réel

Un cabinet RH : chaque matin à 6 h, n8n détecte les nouveaux documents dans son Drive, les indexe avec la date du jour. Les assistantes posent leurs questions par Telegram ; le RAG répond avec sources ; un cron du lundi re-indexe tout. Coût : 4 €/mois de VPS + ~5 € d'API. La règle du cabinet : tout document indexé est relu avant, car il devient « source de vérité » pour l'assistant — une discipline qui vaut mieux que tous les garde-fous techniques.

## Exercice pratique

1. Dessinez l'architecture de VOTRE RAG (quels documents, quels utilisateurs, quel canal).
2. Montez le workflow n8n « Indexation automatique » sur un dossier de test (5 documents).
3. Calculez votre budget mensuel estimé avec les chiffres ci-dessus.
4. Écrivez la liste de vos documents : lesquels contiennent des données personnelles ? Notez la décision de sécurité pour chacun.

## Module 7 — Évaluer son RAG : mesurer, déboguer, améliorer (et ne plus deviner)

### Leçon

Un RAG sans mesure est une croyance. Le passage pro = un jeu de test, des métriques, et un rituel d'amélioration.

1. Construisez votre jeu d'évaluation (la brique n°1) : 20-50 questions réelles, chacune avec :

- la réponse attendue (extrait ou fait vérifiable),
- la source attendue (quel document/paragraphe),
- la catégorie : factuel simple, transverse (multi-sources), négatif (hors documents), terme précis.

2. Les 3 métriques qui comptent :

| Métrique | Question | Comment la mesurer |

|---|---|---|

| Taux de récupération | Le bon chunk est-il dans les top-k ? | % de questions où la source attendue apparaît dans les résultats de recherche (avant le LLM) |

| Taux de réponses correctes | La réponse finale est-elle juste ? | Évaluation manuelle ou par un LLM juge (le modèle B évalue la réponse de A contre la source) |

| Taux de refus corrects | Dit-il « je ne sais pas » quand il le faut ? | % de questions hors documents correctement refusées |

3. Le rituel de débogage en 5 étapes (à exécuter à chaque mauvaise réponse) :

1. La question a-t-elle trouvé le bon chunk ? → sinon : chunking, embeddings, hybride
2. Le bon chunk était-il bien découpé ? → sinon : stratégie de chunking
3. Le chunk était-il dans le top-k ? → sinon : augmenter k, reranking
4. Le prompt a-t-il bien utilisé le chunk ? → sinon : reformuler le prompt
5. Le LLM a-t-il halluciné malgré le contexte ? → sinon : renforcer « je ne sais pas », sources obligatoires

La règle : ne changez qu'une variable à la fois, et rejouez tout le jeu de test après chaque changement. C'est comme ça qu'on passe d'un RAG « qui marche parfois » à un système fiable et démontrable.

4. Le cas « démo » : quand vous montrez votre RAG à un client ou sur les réseaux, montrez les 3 cas qui font la différence : une réponse exacte avec source, une réponse transverse qui assemble 2 documents, et le refus propre (« ce n'est pas dans vos documents ») — c'est ce dernier qui prouve la fiabilité.

Le piège final : le RAG démo parfait sur 3 questions. Tout RAG semble magique sur 3 questions choisies. La crédibilité se construit sur le jeu de test : 30 questions, 90 % de réponses correctes, sources vérifiées. C'est ce chiffre que vous annoncerez — pas une impression.

### Exemple réel

Un commercial construit son jeu de 30 questions sur le manuel produit (20 factuels, 5 transverses, 5

hors docs). V1 : 73 % de correct. Après passage au chunking par sections : 80 %. Après reranking : 87 %. Après renforcement du prompt anti-hallucination : 90 %, dont 5/5 sur les refus. Chaque changement : une variable, une mesure, un chiffre. Il peut maintenant annoncer « 90 % sur notre corpus » avec un jeu de test reproductible derrière.

## Exercice pratique

1. Écrivez 30 questions sur VOS documents (20/5/5 selon les catégories ci-dessus) avec réponses et sources attendues.
2. Faites tourner votre pipeline et mesurez les 3 métriques.
3. Appliquez le rituel de débogage sur vos 5 pires réponses, une variable à la fois.
4. Re-mesurez. Notez le score avant/après dans un fichier eval.md — c'est votre preuve.

## Checklist de fin de formation

- ☐ J'ai identifié 3 documents réels à faire parler à un LLM
- ☐ Je sais expliquer le pipeline RAG en 4 étapes (indexation → stockage → retrieval → génération)
- ☐ Mon script d'embeddings tourne (modèle multilingue, clé en variable d'environnement, jamais en dur)
- ☐ Ma base vectorielle ChromaDB (ou pgvector) stocke mes chunks avec métadonnées (source, date)
- ☐ Mon pipeline complet répond avec sources et dit « je ne sais pas » quand l'info manque
- ☐ Mon chunking est adapté (300-800 car., chevauchement, titre de section embarqué)
- ☐ J'ai testé l'hybride et/ou le reranking sur mes questions à termes précis
- ☐ Mon workflow n8n d'indexation automatique tourne sur un dossier de test
- ☐ Mes coûts mensuels sont chiffrés (< 10 € pour un usage solo)
- ☐ Mes clés API sont protégées (variables d'environnement, passerelle si exposé)
- ☐ Mes données personnelles sont identifiées et sécurisées (RGPD)
- ☐ Mon jeu de test de 30 questions existe, mes 3 métriques sont mesurées (objectif : ≥ 85 % de réponses correctes)

Résultat mesurable : un RAG fonctionnel sur VOS documents, des réponses sourcées, un refus propre hors corpus, un score chiffré — et le pipeline prêt à être branché sur n8n (formation 2) ou vendu comme service (formation 6).

## Prix conseillés

| Formule | Contenu | Prix |

|---|---|---|

| Pack PDF | Les 7 modules + tous les scripts complets (embeddings, ChromaDB, RAG, hybride, reranking) + template de prompt RAG + la checklist d'évaluation | 27 € |

| Pack complet | Pack PDF + le fichier rag.py prêt à l'emploi (indexation + recherche + sources) + le workflow n8n « Indexation automatique » (JSON importable) + le jeu de test de 30 questions (template)

+ le script de mesure des 3 métriques | 47 € |

Note de vente : c'est la formation « infrastructure » du catalogue — elle transforme les LLM (formation 1) et les agents (formation 2) en systèmes qui répondent avec VOS données. Le pack complet s'adresse à celui qui veut un RAG en production cette semaine : les fichiers s'utilisent le jour même, sans repartir de zéro. C'est aussi le socle technique qui justifie une mission « agent avec mémoire de vos docs » (formations 5 et 6) : une compétence rare, démontrable, et facturable.